DraftRev 0

Capacity Scope × Lens View — Decision Tree

Created: 2026-02-24 **Branch:** `feat/capacity-layout` **Purpose:** Map the three scope axes (date range, aggregation level, model mode) against the seven lens views to define what should be shown in every combination — and to reveal gaps in current coverage.

The Three Scope Axes

Every view on the `/capacity` page is the intersection of three independent decisions. Currently, not all three are independently controllable from the UI.

Axis	Options	UI Control Today	Gap?
Axis 1: Date Range	Day · Week · Multi-week · Month+	Filter bar (Start/End)	No gap — user-controlled
Axis 2: Aggregation Level	Daily (per-tick) · Weekly pattern (Mon–Sun avg) · Monthly roll-up	*Implicit* — tied to Forecast lens	**GAP: aggregation is not independent**
Axis 3: Capacity Model	Rotation-based · Headcount-plan-based	*Implicit* — determined by DB state	**GAP: no UI toggle, no comparison mode**

Axis 2 Detail: Aggregation Levels

Level	Chart	X-axis Tick	Value per Tick	Current Trigger
Daily	`CapacitySummaryChart`	1 per calendar day	Actual values for that day	All lenses except Forecast
Weekly Pattern	`ForecastPatternChart`	1 per day-of-week (Mon–Sun)	Average across all matching days in range	Forecast lens only
Monthly roll-up	*(not yet implemented)*	1 per calendar month	Total or average for the month	*(none — missing)*

> **Critical issue:** Aggregation level is **controlled implicitly by the active lens**, not as an independent axis. Selecting "Forecast" both (a) changes the overlay data shown and (b) changes the chart type. These are two separate concerns that have been coupled together.

Axis 3 Detail: Capacity Model Modes

Mode	Computation Path	DB Tables Used	Status
Rotation-based	`resolveStaffingForCapacity()` → `computeDailyCapacityFromStaffing()`	`staffing_configs`, `staffing_shifts`, `rotation_patterns`	**Active** (current DB has config id=1)
Headcount-plan-based	`computeDailyCapacityV2()` with `resolveHeadcount()`	`headcount_plans`, `headcount_exceptions`	Implemented but **inactive** (headcount_plans is empty)
Zero capacity	Fallback when neither path has data	*(none)*	Triggers when both tables empty

> **GAP:** No UI control exists to switch between modes or compare them. The active path is opaque to the user.

Lens View Definitions

Lens	Primary Purpose	Overlay Added	Chart Type	Position in Timeline
Planned	Baseline demand vs capacity	None	`CapacitySummaryChart`	Past + Future
Allocated	Contracted demand vs WP demand	Allocated demand line (contract-based)	`CapacitySummaryChart`	Past + Future
Events	Flight event coverage vs demand	Flight event blocks	`CapacitySummaryChart`	Past + Future
Forecast	Typical weekly pattern + rate forecast	Forecasted demand line; OR Projected MH swap	**`ForecastPatternChart`**	Past (avg) + Future (projected)
Worked	Actual hours vs demand and capacity	Worked hours line (time_bookings)	`CapacitySummaryChart`	**Past only**
Billed	Billable hours vs demand and capacity	Billed hours line (billing_entries)	`CapacitySummaryChart`	**Past only**
Concurrent	Concurrency pressure on capacity	Peak concurrency line	`CapacitySummaryChart`	Past + Future (estimated)

Decision Tree

```
ENTRY: User opens /capacity page
│
├── AXIS 1: What time window am I looking at?
│   ├── Short range (≤ 7 days) ──────────────────────────────────┐
│   ├── Standard range (1–4 weeks) ─────────────────────────────┤ → proceed
│   ├── Long range (1–3 months) ────────────────────────────────┤
│   └── Extended range (3+ months) ─────────────────────────────┘
│
├── AXIS 2: What level of aggregation do I need?
│   ├── Daily (exact dates) ──────────────────────────── → CapacitySummaryChart
│   │   └── Proceeds to Axis 3 + Lens selection
│   ├── Weekly pattern (Mon–Sun averages) ──────────────── → ForecastPatternChart
│   │   └── Lens is FORCED to "Forecast" (current design)
│   │   └── [GAP] Should be independent of lens
│   └── Monthly roll-up [NOT IMPLEMENTED]
│       └── [GAP] No chart component or aggregation engine exists
│
├── AXIS 3: Which capacity model is active?
│   ├── Rotation-based (active) ──────────── capacity from staffing_shifts + rota
│   ├── Headcount-plan-based (inactive) ─── capacity from headcount_plans + exce
│   └── [GAP] No way to see WHICH is active, or compare both
│
└── LENS: What insight layer do I want?
    │
    ├── PLANNED ──────────────────────────────────────────────────────
    │   Chart: CapacitySummaryChart (daily)
    │   Bars: totalDemandMH (or per-shift, or per-customer)
    │   Lines: productiveMH (capacity, dashed), utilizationPercent (solid, right
    │   Overlay: None
    │   Use when: "What is my raw demand vs capacity baseline?"
    │   Works for: Past ✓ | Future ✓ | By Shift ✓ | By Customer ✓
    │
    ├── ALLOCATED ────────────────────────────────────────────────────
    │   Chart: CapacitySummaryChart (daily)
    │   Bars: totalDemandMH
    │   Lines: capacity (dashed), utilization (solid), allocatedDemandMH (accent
    │   Overlay: demand_contracts → demand_allocation_lines aggregated by date+s
    │   Use when: "Are contracted demand allocations in line with actual WP dema
```

Full Combination Matrix

For each Lens × Aggregation × Time Position combination, what can currently be shown:

Lens	Daily Timeline	Weekly Pattern	Monthly Roll-up	Past	Future	By Shift	By Customer
Planned	✅	❌ (not accessible)	❌ missing	✅	✅	✅	✅
Allocated	✅	❌	❌ missing	✅	✅	✅	✅
Events	✅	❌	❌ missing	✅	✅	✅	❌ events not per-customer
Forecast	❌ (no daily forecast line)	✅	❌ missing	✅ (avg)	✅ (projected)	✅	✅
Worked	✅	❌	❌ missing	✅	❌ no future data	✅	✅
Billed	✅	❌	❌ missing	✅	❌ no future data	✅	✅
Concurrent	✅	❌	❌ missing	✅	✅ (est.)	✅	❌ not per-customer

Gaps visible in matrix:

- All lenses are missing from **Weekly Pattern** except Forecast
- **Monthly Roll-up** does not exist for any lens
- **Forecast** is missing from the **Daily Timeline** (no daily forecasted demand line)
- **Worked** and **Billed** have no future-date behavior defined

Gap Inventory

G-01: Aggregation level is coupled to the Forecast lens

Current: Selecting the Forecast lens is the only way to access the weekly pattern chart. Changing the aggregation level is not a separate user action. **Impact:** Users cannot view Planned / Allocated / Worked in weekly-pattern form. **Resolution option:** Decouple aggregation (Daily | Weekly Pattern | Monthly) into an independent control, separate from the lens selector. The lens controls *what overlay data* is shown; the aggregation controls *how time is bucketed*.

G-02: No daily forecast line

Current: The rate-based forecast (`avgForecastedDemandMH`) is only accessible via the weekly pattern chart. **Impact:** Users cannot see "what the forecast model predicts for each specific date" on the daily timeline. **Resolution option:** Add a `forecastedDemandMH` line to `CapacitySummaryChart` when Forecast lens is active, alongside the existing bars.

G-03: Weekly pattern averages are scope-limited

Current: `computeDayOfWeekPattern()` averages only the dates in the current filter window. **Impact:** A 1-week filter gives 1-sample averages (not statistically meaningful as a "typical" pattern). **Resolution option:** Add a "history window" control separate from the date filter, or always use full available history for the pattern computation.

G-04: Projected toggle not accessible from daily timeline

Current: The "Projected" toggle (weekly MH projections from `weekly_mh_projections`) is only accessible in the Forecast lens / weekly pattern chart. **Impact:** Users cannot see projected demand bars on the daily timeline view. **Resolution option:** Allow projections to expand/fill the daily timeline for future dates (e.g., repeat the day-of-week projected value for each future date).

G-05: No model mode visibility or toggle

Current: Whether the app is using rotation-based or headcount-plan-based capacity is invisible to the user. **Impact:** If configuration changes (e.g., staffing config is deactivated), capacity silently drops to zero with no explanation. **Implementation note:** `computeMode` and `activeStaffingConfigName` are **already in the Zustand store** (`use-capacity-v2.ts`) and returned by `/api/capacity/overview` . The data is available — it is simply not rendered anywhere. This is a small UI-only change.

Resolution option: Add a status badge near the KPI strip or page header showing the active compute mode (e.g. “Rotation: Default Configuration”) using `activeStaffingConfigName` from the store. `resolvedShifts` is also available for per-shift detail.

G-06: Worked/Billed have no future-date behavior

Current: When the date range extends into the future, Worked and Billed lenses show lines that end abruptly at today with no explanation. **Impact:** Confusing visual gap — unclear whether data is missing or the lens is future-incompatible. **Resolution option:** Add a visual “today” reference line and a note indicating “actuals available up to [date]”. Optionally shade future dates differently.

G-07: No cross-lens comparison

Current: Only one lens can be active at a time. **Impact:** Cannot compare Worked vs Allocated vs Planned in a single view. **Resolution option:** Allow up to 2 lenses simultaneously (primary + secondary overlay), shown as two accent lines with different colors.

G-08: No gap analysis view

Current: Utilization % is shown, but not the raw surplus/deficit in MH. **Impact:** “How many MH do I have spare / how many am I short?” requires mental arithmetic. **Resolution option:** Add a “Gap” view mode (alongside Total / By Shift / By Customer) that shows `capacity - demand` as a bar — positive = surplus (green), negative = deficit (red).

G-09: Monthly roll-up aggregation missing entirely

Current: No monthly aggregation exists anywhere in the system. **Impact:** Cannot answer “what was our average utilization in January?” without exporting data. **Resolution option:** Add a third aggregation level (Monthly) that shows 1 bar per calendar month, averaged across all days in that month.

G-10: Events lens not per-customer

Current: Flight events are shift-level only — no customer attribution. **Impact:** Cannot determine which customer’s flights are driving concurrency or coverage requirements. **Resolution option:** Add customer field to flight_events and surface in By Customer view mode.

Priority Tiers for Gap Resolution

Tier 1 — High impact, relatively contained

Gap	Effort	Value
G-01: Decouple aggregation from lens	Medium	High — unlocks all lenses on weekly pattern
G-02: Daily forecast line	Low	High — makes Forecast lens useful on daily chart
G-03: History window for pattern	Low	Medium — improves statistical accuracy
G-06: Today line + future shading	Low	Medium — eliminates confusing visual gaps

Tier 2 — Medium complexity, significant value

Gap	Effort	Value
G-04: Projections on daily timeline	Medium	High — surfaces weekly projections where most useful
G-05: Model mode visibility	Low	High — operational transparency
G-08: Gap analysis view mode	Medium	High — direct answer to “how short am I?”

Tier 3 — Larger scope changes

Gap	Effort	Value
G-07: Cross-lens comparison	High	Medium — useful but complex UI
G-09: Monthly roll-up	High	Medium — new aggregation engine needed
G-10: Events per-customer	Medium	Medium — requires schema change

---

Recommended Immediate Action: G-01 + G-02

Decoupling aggregation from lens (G-01) is the architectural unlock that makes all other improvements possible. Without it, every new lens stays siloed to the daily chart.

Proposed new UI layout:

```
/capacity page header:
  ┌─ Filter bar (dates, operator, aircraft, type) ──────────────────────┐
  │                                                                      │
  │  LENS: [Planned] [Allocated] [Events] [Forecast] [Worked] [Billed] [Concurre
  │                                                                      │
  │  VIEW: [Daily Timeline] [Weekly Pattern] [Monthly Roll-up]          │
  │                                                  ↑ new independent control
  └──────────────────────────────────────────────────────────────────────┘


  CHART: determined by VIEW selection, not by LENS
  OVERLAY: determined by LENS selection, not by VIEW
```

Implementation note (from page.tsx): The current chart switch is a single ternary at line ~200:

```
{activeLens === "forecast" ? <ForecastPatternChart ... /> : <CapacitySummaryChart ... />}
```

Replacing this with a `viewAggregation` state variable (`"daily" | "weekly" | "monthly"`) and keeping lens as a separate selector is the minimal, targeted change. The `LensSelector` component already exists as the natural place to add the aggregation control alongside it.

This change:

1. Keeps all existing lens behavior
2. Makes `ForecastPatternChart` accessible from any lens
3. Removes the implicit “Forecast lens → switch chart” behavior
4. Opens the path for G-02 (Forecast overlay on daily chart becomes the standard Forecast lens behavior on daily view)

Enhancement Plan: Forecasting, Scenarios & Gap Analysis

> **Objective:** Make the capacity model actionable for management by adding forward-looking trend lines, what-if scenarios, and clear surplus/deficit metrics — all integrated into the existing lens + chart architecture.

E-01: Rolling 8-Week Forecast Line

What: A trend-based forecast line overlaid on the `CapacitySummaryChart` (daily timeline) that projects demand forward 8 weeks based on historical day-of-week patterns weighted by recency.

Engine: `computeRolling8WeekForecast()` (new, pure function)

- Input: `demand: DailyDemandV2[]`, `capacity: DailyCapacityV2[]`, `forecastWeeks: number` (default 8)
- Algorithm:
  1. Compute day-of-week averages from historical demand (same logic as `computeDayOfWeekPattern()` but uses all available data, not just the filter window — addresses G-03)
  2. Apply exponential decay weighting: more recent weeks contribute more heavily than older weeks (decay factor ~0.85 per week)
  3. Generate one `ForecastPoint` per calendar day for the next `forecastWeeks × 7` days
  4. Each point: `{ date, forecastedDemandMH, forecastedByShift, confidence: "high" | "medium" | "low" }` where confidence degrades as the horizon extends
- Output: `RollingForecastResult { points: ForecastPoint[], horizonStart: string, horizonEnd: string, basedOnWeeks: number }`

Chart integration:

- Rendered as a **dotted line** (`strokeDasharray="4 4"`) in a distinct color (e.g., `#a855f7` purple) on `CapacitySummaryChart`
- Line extends beyond the historical demand bars into the future date range
- Tooltip shows: "8-Week Forecast: X MH (based on Y weeks of data)"
- In `byShift` mode: 3 per-shift forecast lines
- In `total` mode: 1 aggregate forecast line
- Controlled by a toggle button in the chart header (similar to existing "Projected" toggle on ForecastPatternChart)

Relationship to existing gaps:

- Addresses G-02 (daily forecast line) and G-03 (history window for averaging) simultaneously
- Uses the same `computeDayOfWeekPattern` fix (capacity independent of demand dates) we just applied

---

E-02: Scenario Toggle — Baseline vs +10% Demand

What: A scenario selector that lets users compare the current baseline with a "+10% demand increase" scenario. The scenario applies a multiplier to demand values (and optionally adjusts capacity to model different staffing responses).

Scenarios defined:

| Scenario | Demand Multiplier | Capacity Adjustment | Use Case | |----------|----------|----------|---------|| | **Baseline** | 1.0× | None | Current state — default | | **+10% Demand** | 1.1× | None | "What if demand grows 10% — are we still covered?" | | **(Future: Custom)** | User-defined | User-defined | Extensible for arbitrary what-if modeling |

Engine: `applyScenario()` (new, pure function)

- Input: `demand: DailyDemandV2[]`, `capacity: DailyCapacityV2[]`, `scenario: ScenarioConfig`
- `ScenarioConfig: { id: string, label: string, demandMultiplier: number, capacityMultiplier: number }`
- Logic: deep-copies demand/capacity arrays, applies multipliers to all MH fields (total and per-shift), recomputes utilization
- Output: `ScenarioResult { demand, capacity, utilization, gapMetrics }` (same shape as baseline but scaled)
- **Key constraint:** This is a pure client-side transformation — no DB changes, no API changes. The scenario engine takes the already-computed baseline data and applies multipliers.

Predefined scenarios (hardcoded initially, admin-configurable later):

```
const SCENARIOS: ScenarioConfig[] = [
  { id: "baseline", label: "Baseline", demandMultiplier: 1.0, capacityMultiplier: 1.0 },
  { id: "demand_10", label: "+10% Demand", demandMultiplier: 1.1, capacityMultiplier: 1.0 },
];
```

E-03: Gap Metrics Per Scenario

What: KPI cards and per-day gap values showing surplus/deficit in MH for each active scenario.

Gap computation: `computeGapMetrics()` (new, pure function)

- Input: `demand: DailyDemandV2[]`, `capacity: DailyCapacityV2[]`, `utilization: DailyUtilizationV2[]`
- Output per day:

```
interface DailyGapMetrics {
  date: string;
  totalGapMH: number;         // capacity - demand (positive = surplus, negative = defici
  gapByShift: Record<string, number>;  // per shift: capacity - demand
  gapPercent: number;          // totalGapMH / totalCapacityMH × 100
  status: "surplus" | "balanced" | "deficit" | "critical";
  // surplus: gap > 20%, balanced: -20% to +20%, deficit: -20% to -50%, critical: < -50%
}
```

- Aggregate summary:

```
interface ScenarioGapSummary {
  scenarioId: string;
  scenarioLabel: string;
  avgGapMH: number;           // average daily gap across the range
  totalDeficitDays: number;   // days where gap < 0
  totalCriticalDays: number;  // days where gap < -50%
  worstDay: { date: string; gapMH: number } | null;
  bestDay: { date: string; gapMH: number } | null;
  netGapMH: number;           // sum of all daily gaps
}
```

Chart integration (addresses G-08):

- New view mode option: **Total | By Shift | By Customer | Gap** in the chart header toggle
- Gap mode renders as a diverging bar chart:
  - Positive bars (surplus) extend upward — colored green (`#22c55e`)
  - Negative bars (deficit) extend downward — colored red (`#ef4444`)
  - Zero line clearly visible
  - In `byShift` mode within Gap view: 3 grouped diverging bars
- When scenario toggle is active, the gap chart shows **both** scenarios side-by-side (baseline gap as solid bars, scenario gap as outlined/semi-transparent bars)

KPI Strip integration:

- When any scenario is active (not baseline), KPI strip shows a comparison row:

```
┌──────────────────┬──────────────────┬──────────────────┐
│ AVG GAP          │ DEFICIT DAYS     │ WORST DAY        │
│ Baseline: +12 MH │ Baseline: 3      │ Baseline: Feb 20 │
│ +10%:     -8 MH  │ +10%:     7      │ +10%:     Feb 20 │
│ ▼ -20 MH change  │ ▲ +4 days        │ Gap: -45 MH      │
└──────────────────┴──────────────────┴──────────────────┘
```

E-04: Scenario Controls UI Integration

What: Seamless integration of scenario selection and forecast display into the existing dashboard layout.

UI placement:

```
/capacity page — enhanced layout:
  ┌──────────────────────────────────────────────────────────────────┐
  │  LENS: [Planned] [Allocated] [Events] [Forecast] [Worked] ...    │
  │                                                                  │
  │  SCENARIO: ( Baseline ) ( +10% Demand )     [8W Forecast: ON/OFF] │
  │            ↑ radio toggle                    ↑ toggle button     │
  └──────────────────────────────────────────────────────────────────┘
```

Component: `ScenarioSelector` (new component)

- Radio-button group (not tabs — because only 1 scenario is active at a time for chart rendering, but gap metrics show both)
- Placed below the `LensSelector` row, above the KPI strip
- Visual: pill-style radio buttons matching the existing lens selector pattern
- Active scenario highlighted with accent color
- "8-Week Forecast" toggle button positioned at the right end of the scenario row

State management:

- New Zustand state in `use-capacity-v2.ts`:

```
activeScenario: ScenarioConfig;          // currently selected scenario
showForecastLine: boolean;               // toggle for 8-week forecast line
setActiveScenario: (s: ScenarioConfig) => void;
setShowForecastLine: (v: boolean) => void;
```

- Scenario application is **client-side only** — no API changes needed. The store holds baseline data from the API, and derived scenario data is computed via `useMemo` in the page component.

Data flow with scenarios:

```
API Response (baseline) ──→ Zustand Store (raw baseline data)
                                        │
                                        ├──→ applyScenario(baseline, activeScenario)
                                        │
                                        ├──→ computeGapMetrics(baseline) ──→ baseli
                                        │
                                        └──→ computeGapMetrics(scenario) ──→ scenar

                                        ├──→ computeRolling8WeekForecast(baseline) -

                                        └──→ Charts + KPI Strip receive both baseli
```

Key design principles:

1. **Zero API changes** — all scenario/forecast computation is client-side over already-fetched data
2. **Non-destructive** — baseline data is always preserved; scenarios derive from it
3. **Composable** — scenarios work with all existing lenses (the lens overlay is applied after the scenario multiplier)
4. **Management-friendly** — gap metrics use plain language (surplus/deficit/critical), not jargon

---

Enhancement Summary Table

Enhancement	Addresses Gaps	New Engine Function	New Component	API Change	Priority
E-01: 8-Week Forecast Line	G-02, G-03	`computeRolling8WeekForecast()`	Toggle button	None	High
E-02: Scenario Toggle	*(new)*	`applyScenario()`	`ScenarioSelector`	None	High
E-03: Gap Metrics	G-08	`computeGapMetrics()`	Gap KPI cards, Gap view mode	None	High
E-04: Scenario Controls	*(new)*	—	UI integration	Store additions	High

Implementation Order

1. **E-03 first** (gap metrics engine + view mode) — foundational, no UI dependencies
2. **E-01 second** (forecast engine + chart line) — standalone engine + chart overlay
3. **E-02 third** (scenario engine) — depends on gap metrics for its output
4. **E-04 last** (UI integration) — wires everything together

Files to Create/Modify

File	Action	Purpose
`src/lib/capacity/scenario-engine.ts`	**Create**	`applyScenario()`, `ScenarioConfig`, `SCENARIOS`
`src/lib/capacity/gap-engine.ts`	**Create**	`computeGapMetrics()`, `DailyGapMetrics`, `ScenarioGapSummary`
`src/lib/capacity/rolling-forecast-engine.ts`	**Create**	`computeRolling8WeekForecast()`, `ForecastPoint`, `RollingForecastResult`
`src/lib/capacity/index.ts`	Modify	Export new engines from barrel
`src/types/index.ts`	Modify	Add `ScenarioConfig`, `DailyGapMetrics`, `ScenarioGapSummary`, `ForecastPoint`, `RollingForecastResult`
`src/components/capacity/scenario-selector.tsx`	**Create**	Scenario radio toggle + forecast line toggle
`src/components/capacity/gap-kpi-cards.tsx`	**Create**	Surplus/deficit comparison cards
`src/components/capacity/capacity-summary-chart.tsx`	Modify	Add forecast line rendering + gap view mode
`src/components/capacity/capacity-kpi-strip.tsx`	Modify	Add gap metrics display when scenario active
`src/lib/hooks/use-capacity-v2.ts`	Modify	Add `activeScenario`, `showForecastLine` state
`src/app/(authenticated)/capacity/page.tsx`	Modify	Wire ScenarioSelector, pass scenario data to charts
`src/__tests__/capacity/gap-engine.test.ts`	**Create**	Gap metrics tests
`src/__tests__/capacity/scenario-engine.test.ts`	**Create**	Scenario application tests
`src/__tests__/capacity/rolling-forecast-engine.test.ts`	**Create**	Forecast computation tests

Open Items Raised by This Analysis

Structural Gaps

ID	Item	Priority
OI-new	G-01: Decouple aggregation level from lens selection	High
OI-new	G-02: Add daily forecast demand line to CapacitySummaryChart	High — **subsumed by E-01**
OI-new	G-03: Add “history window” option for weekly pattern averaging	Medium — **subsumed by E-01**
OI-new	G-06: Add today reference line and future-date shading	Medium
OI-new	G-08: Add “Gap” view mode (capacity − demand surplus/deficit)	Medium — **subsumed by E-03**
OI-new	G-04: Surface weekly projections on daily timeline for future dates	Medium
OI-new	G-05: Show active capacity model mode in UI	Low

Enhancements (Management Actionability)

ID	Item	Priority	Blocks / Blocked By
OI-new	E-01: Rolling 8-week forecast line on daily chart	High	Standalone
OI-new	E-02: Scenario toggle (baseline vs +10% demand)	High	Depends on E-03
OI-new	E-03: Gap metrics per scenario (surplus/deficit)	High	Standalone — implement first
OI-new	E-04: Scenario controls UI integration	High	Depends on E-01, E-02, E-03